

1 Soluție — Proeminența

1.1 Descrierea problemei

Se citește un șir de N altitudini și un parametru C . Pentru $C = 1$ trebuie numărate vârfurile (puncte strict mai înalte decât vecinii). Pentru $C = 2$ trebuie calculată proeminența fiecărui vârf, definită ca diferența dintre înălțimea vârfului și cea mai adâncă vale din stânga sau dreapta sa.

1.2 Ideea principală

Observăm că proeminența unui vârf depinde doar de prima vale mai adâncă din stânga și din dreapta. Putem procesa șirul de la stânga la dreapta, menținând o stivă cu vârfurile întâlnite în ordine strict descrescătoare și, pentru fiecare vârf din stivă, cea mai adâncă vale întâlnită după el.

Când întâlnim un vârf nou mai înalt decât vârful din vârful stivei, acesta elimină toate vârfurile mai mici sau egale — deci stiva rămâne întotdeauna ordonată strict descrescător.

Pentru a calcula proeminența la dreapta, oglindim șirul și reaplicăm același algoritm.

1.3 Algoritmul

1. Se citesc datele și se elimină duplicatele consecutive.
2. Pentru $C = 1$: se numără pozițiile i cu $a[i - 1] < a[i] > a[i + 1]$.
3. Pentru $C = 2$:
 - Se parcurge șirul de la stânga la dreapta, construind stiva de vârfuri.
 - Când se întâlnește un vârf, se elimină din stivă toate vârfurile cu înălțime mai mică.
 - Dacă stiva nu e goală, proeminența la stânga este $a[\text{peak}] - \text{valey}[\text{top}]$.
 - Se actualizează continuu cea mai adâncă vale pentru vârful curent din stivă.
 - Se oglindește șirul și se repetă pentru calculul proeminenței la dreapta.
 - Pentru fiecare vârf, proeminența finală este minimumul dintre cele două valori calculate.

Complexitate: $O(N)$ timp și $O(N)$ memorie.

1.4 Codul sursă

```

#include <stdio.h>
#define Nmax 1000005
long N, C, s;
long a[Nmax];

long min_(long i, long j) { if (i < j) return i; return j; }
long peak[Nmax], valey[Nmax], top = -1;
long prom[2][Nmax];

void popTop() {
    if (top < 1) { return; }
    if (top >= 1 && valey[top-1] > valey[top]) {
        valey[top-1] = valey[top];
    }
    top--;
}

int main() {
    freopen("proeminenta.in", "r", stdin);
    freopen("proeminenta.out", "w", stdout);

    scanf("%ld_%ld", &C, &N);
    for (int i = 0; i < N; i++) {
        scanf("%ld", &a[i]);
        if (i > 0 && a[i] == a[i-1]) {
            i--; N--;
        }
    }
    if (C == 1) {
        for (int i = 1; i < N-1; i++) {
            if (a[i-1] < a[i] && a[i] > a[i+1])
                { s++; }
        }
        printf("%ld\n", s);
        return 0;
    }

    for (int d = 0; d <= 1; d++) {
        for (int i = 1; i < N-1; i++) {
            if (a[i-1] < a[i] && a[i] > a[i+1])
            {
                prom[d][i] = a[i];
                while (top >= 0 && a[peak[
                    top]] < a[i]) {
                    popTop();
                }
                if (top >= 0 && a[peak[top]]
                    > a[i]) { prom[d][i] = a
                        [i] - valey[top]; }
                peak[++top] = i;
            }
        }
    }
}

```

```

        valey[top] = a[i];
    }
    if (top > 0 && valey[top] > a[i]) {
        valey[top] = a[i]; }
    }
    for (int i = 0; i < N / 2; i++) {
        long aux = a[i]; a[i] = a[N-1-i]; a[
            N-1-i] = aux;
    }
    top = -1;
}
for (int i = 1; i < N-1; i++) if (a[i-1] < a[i] && a
    [i] > a[i+1]) {
    if (prom[0][i] > prom[1][N-1-i]) {
        prom[0][i] = prom[1][N-1-i];
    }
}

for (int i = 1; i < N-1; i++) {
    if (prom[0][i] > 0) { printf("%ld_", prom
        [0][i]); }
}

printf("\n");
fclose(stdout);
return 0;
}

```

1.5 Observații importante

- Stiva menține vârfurile în ordine strict descrescătoare, ceea ce asigură că proeminența se calculează corect.
- Ogindirea șirului permite reutilizarea aceluiași cod pentru ambele direcții.
- Dimensiunea array-ului $N_{\max} = 1000005$ este suficientă pentru restricțiile problemei.